scai

SORBONNE CLUSTER FOR
ARTIFICIAL INTELLIGENCE

Quiz

1. How to manage missing values?
2. What are the different categorical values and how to manage them?
3. What is an outlier?
4. What is scaling data? Why are we doing it?
4. Examples of feature engineering?

Course 4 DS — Neural Networks

Raphael Cousin
Data Scientist
raphael.cousin@sorbonne-universite.fr

raphaelcousin.com

scai
SORBONNE CLUSTER FOR ARTIFICIAL INTELLIGENCE

Plan

From Linear to Non-Linear

Duration : ~10 min

Multi-Layer Perceptron

Duration : ~20 min

Architectures

Duration : ~10 min

Training

Duration : ~10 min

Practical Considerations

Duration : ~10 min

MLP in scikit-learn

Duration : ~10 min

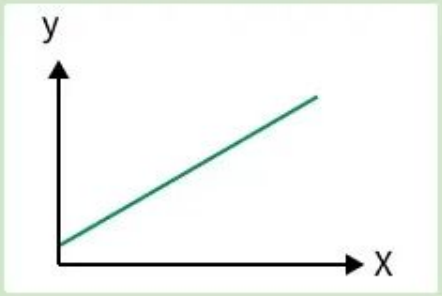
scai
SORBONNE CLUSTER FOR ARTIFICIAL INTELLIGENCE

From Linear to Non-Linear

Linear Regression -> regression

Linear Regression

- Predicts continuous values
- Uses best-fit line
- Solves regression problems



$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p = X\theta$$

Logistic Regression -> Classification

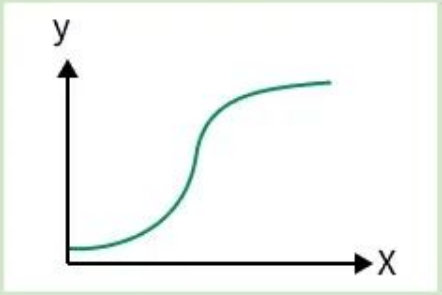
$$P(Y = 1 \mid x) = \sigma(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

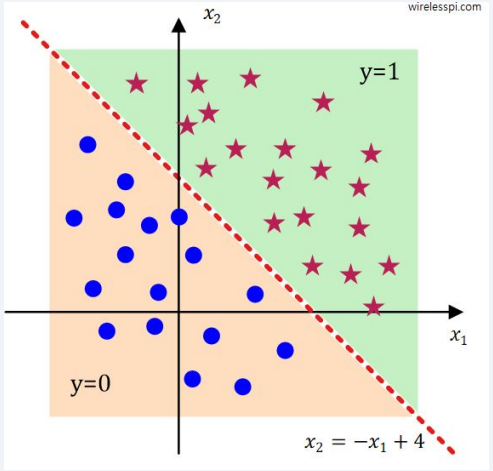
$$\hat{y} = \{1 \text{ si } \theta^T x > 0 \; 0 \text{ si } \theta^T x < 0\}$$

$$\hat{y} = \{1 \text{ si } P(Y=1|x) \geq 0.5 \; 0 \text{ sinon}\}$$

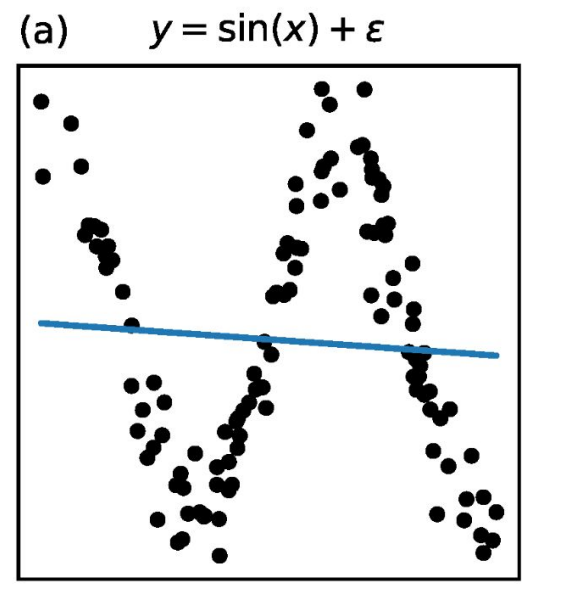
Logistic Regression

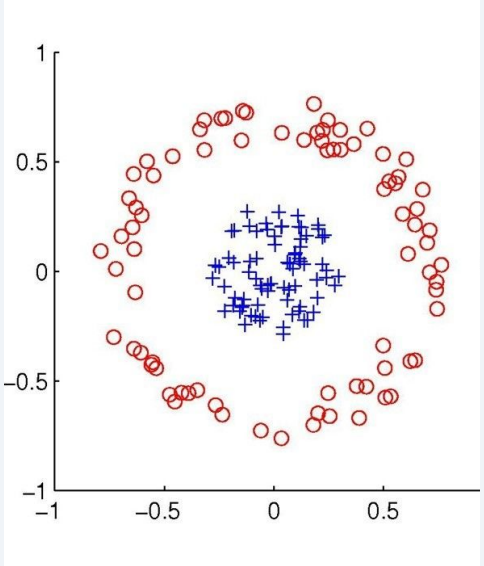
- Predicts categorical classes
- Uses sigmoid S-curve
- Solves classification problems





Limits of Linear Models

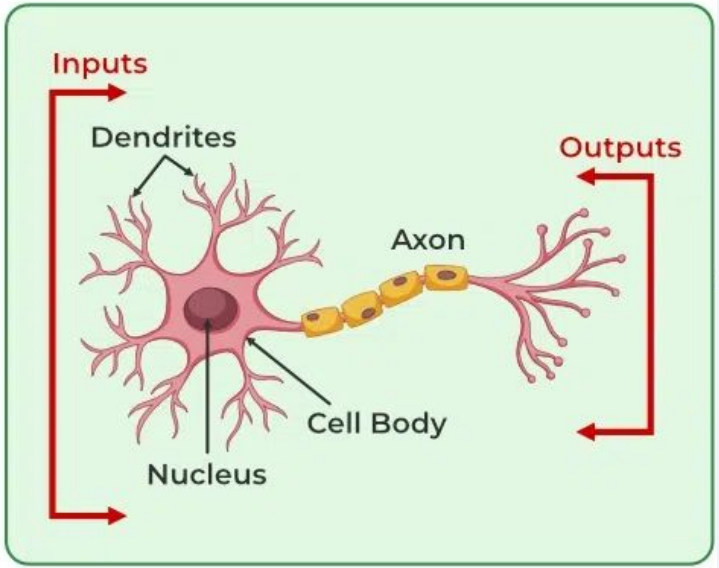




Linear Regression and logistic regression will not cover those cases

Multi-Layer Perceptron

The Biological Neuron



The Artificial Neuron

Input: $X = (x_1, \ldots, x_{r_0})$

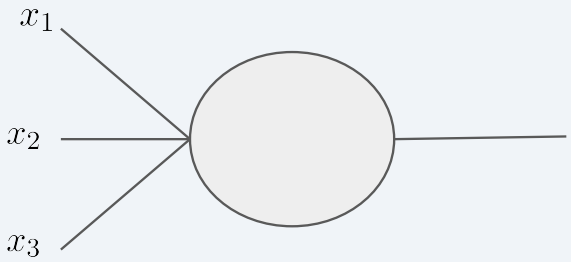
Params: dimension of input + biais

$$w = (b, w_1, \ldots, w_{r_0})$$

Operation:

Activation $o = \sigma(w \cdot X)$

Output: dimension 1



1 neuron

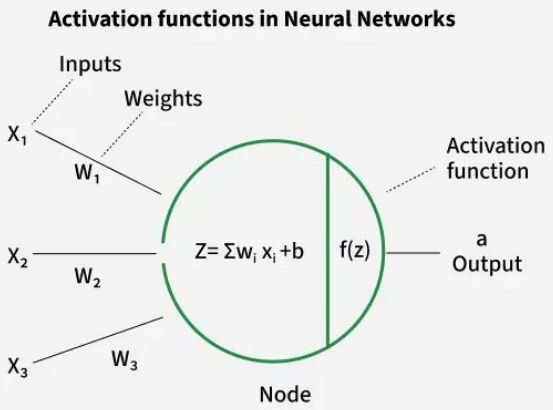
$$z = \sum_i w_i x_i + b$$

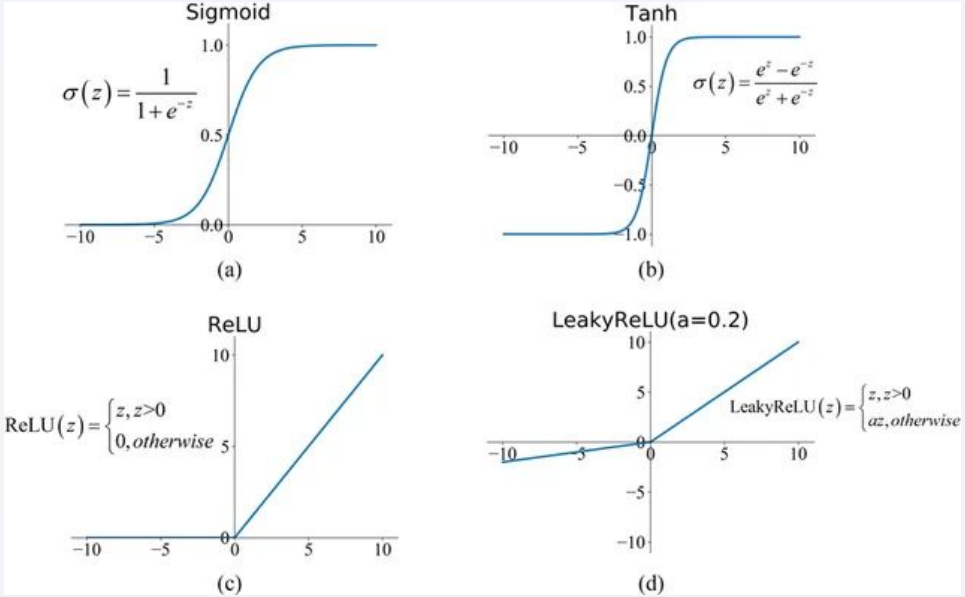
$$\sigma(z)$$

To notice:

With activation identity, neuron is the same as linear regression

Activation Functions





Multi-Layer Perceptron (MLP)

A model like any other

Input: $X = (x_1, \ldots, x_{r_0})$

Parameters:

$W^1, \ldots, W^l \qquad W^k \in \mathbb{R}^{r_{k-1} \times r_k}$

l Layers

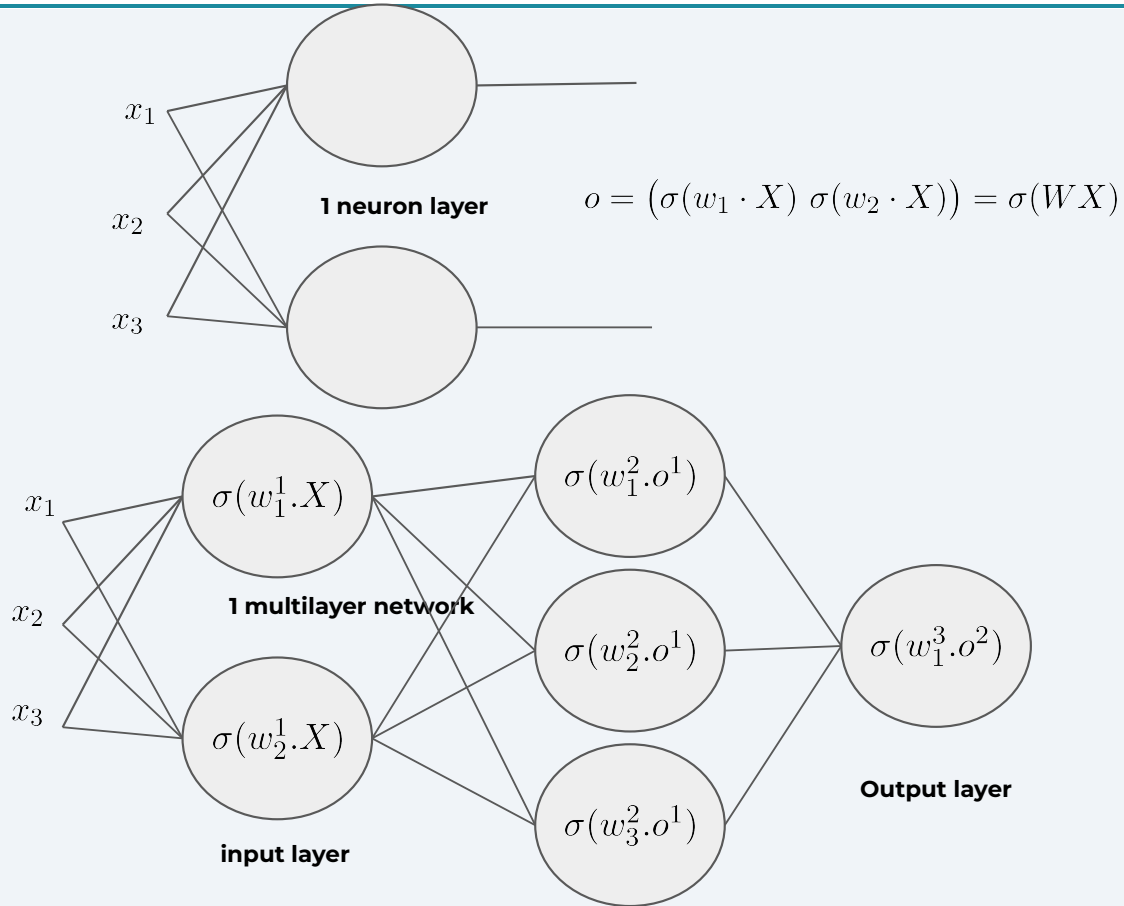
r_k neurons per layer

$\text{total} = \sum_{k=1}^{l} r_k \cdot (r_{k-1} + 1)$

Operation:

$f_\theta(X) = W^l \sigma(W^{l-1} \sigma(\ldots \sigma(W^1 X)))$

Output: number of neuron in last layer



Quiz

1. Combien de paramètre a apprendre dans un MLP avec dimension de X=3, 1 couche cachée de taille 5, une couche caché de taille 3, et une couche de sortie de taille 1?
2. Quelle est la formule générale $X = (x_1, \ldots, x_{r_0})$ l Layers r_k neurons per layer
3. Il y a t-il un intérêt à faire suivre 2 couches sans fonction d'activation?

Quiz

2.

Poids : $W^k \in \mathbb{R}^{r_{k-1} \times r_k} \to r_{k-1} \cdot r_k$

Biais : $b^k \in \mathbb{R}^{r_k} \to r_k$

$$\text{total} = \sum_{k=1}^{l} r_k \cdot (r_{k-1} + 1)$$

Total par couche : $r_k(r_{k-1} + 1)$

1. $r_0 = 3 \to r_1 = 5 \to r_2 = 3 \to r_3 = 1$

$= r_1(r_0 + 1) + r_2(r_1 + 1) + r_3(r_2 + 1) = 5 \times 4 + 3 \times 6 + 1 \times 4 = 20 + 18 + 4 = \boxed{42}$

3. $W^2(W^1X + b^1) + b^2 = \underbrace{W^2W^1}X + \underbrace{W^2b^1 + b^2} = W'X + b'$

Training Neural Networks

Same objective and training as other parametric model :

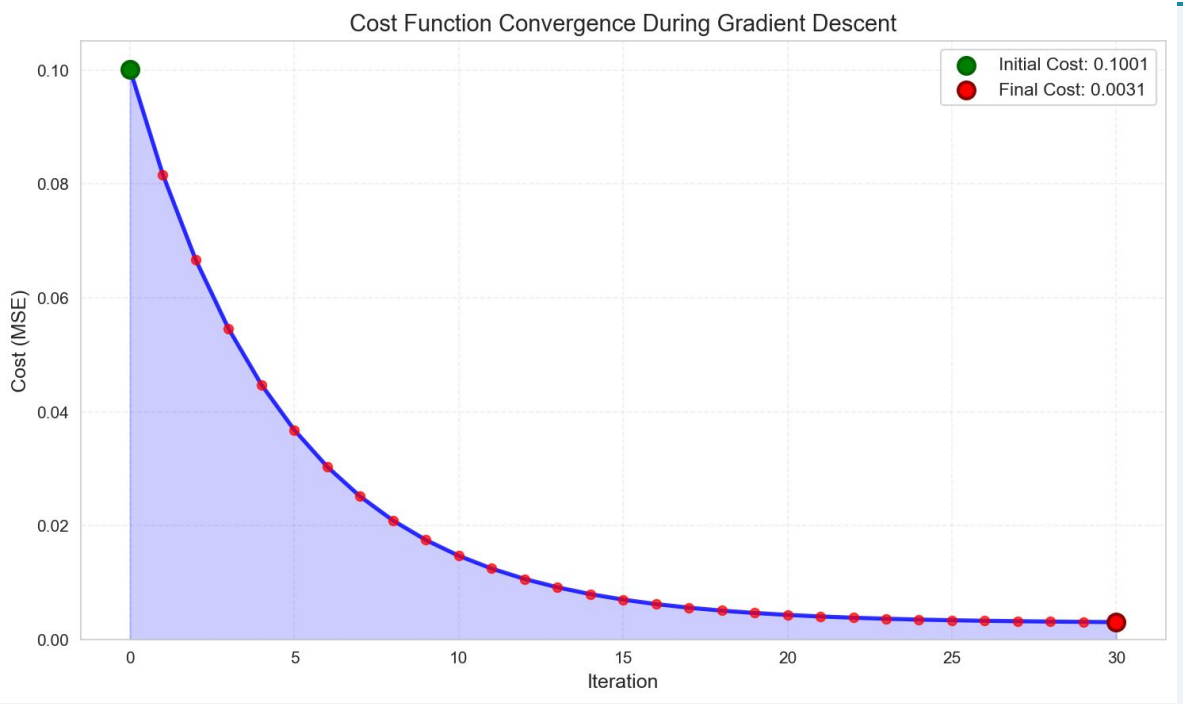
0. initialise the weights randomly

Train:

1. Compute the gradient of the loss
2. Update the parameters using gradient descent
3. Iterate

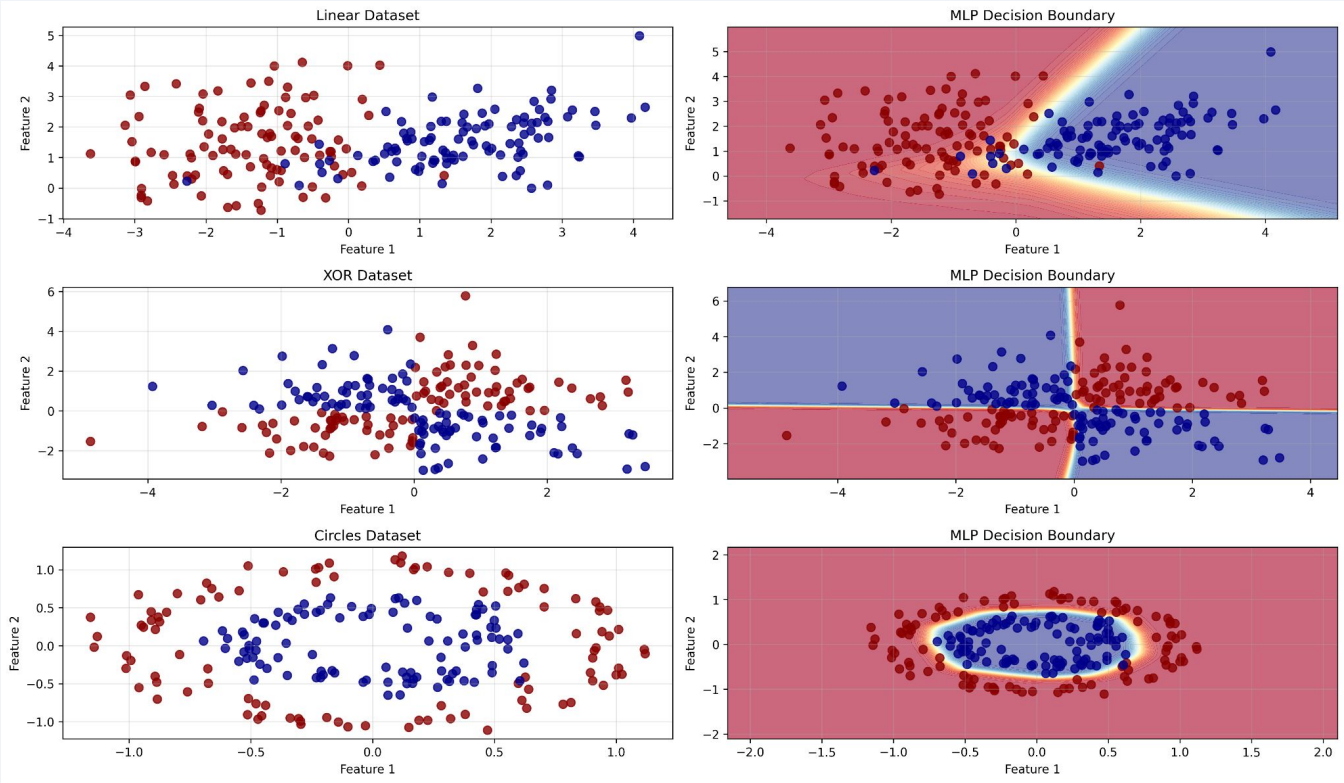
Predict:

Use the model to predict



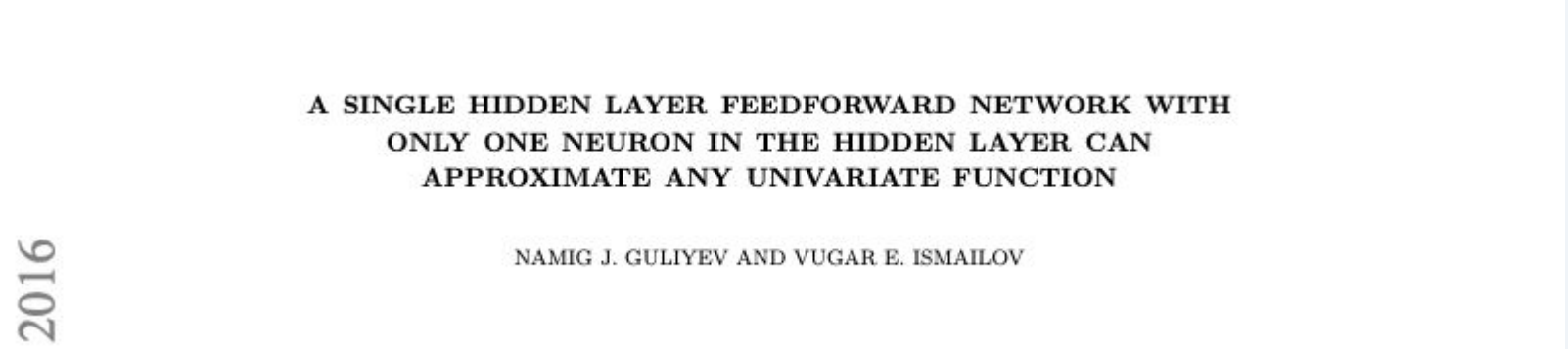
$$\arg\min_{\theta \in \mathbb{R}^p} \ell(Y, f_\theta(X))$$

Solve Nonlinear Problem



Architectures

Universal Approximation Theorem



A SINGLE HIDDEN LAYER FEEDFORWARD NETWORK WITH ONLY ONE NEURON IN THE HIDDEN LAYER CAN APPROXIMATE ANY UNIVARIATE FUNCTION

NAMIG J. GULIYEV AND VUGAR E. ISMAILOV

2016

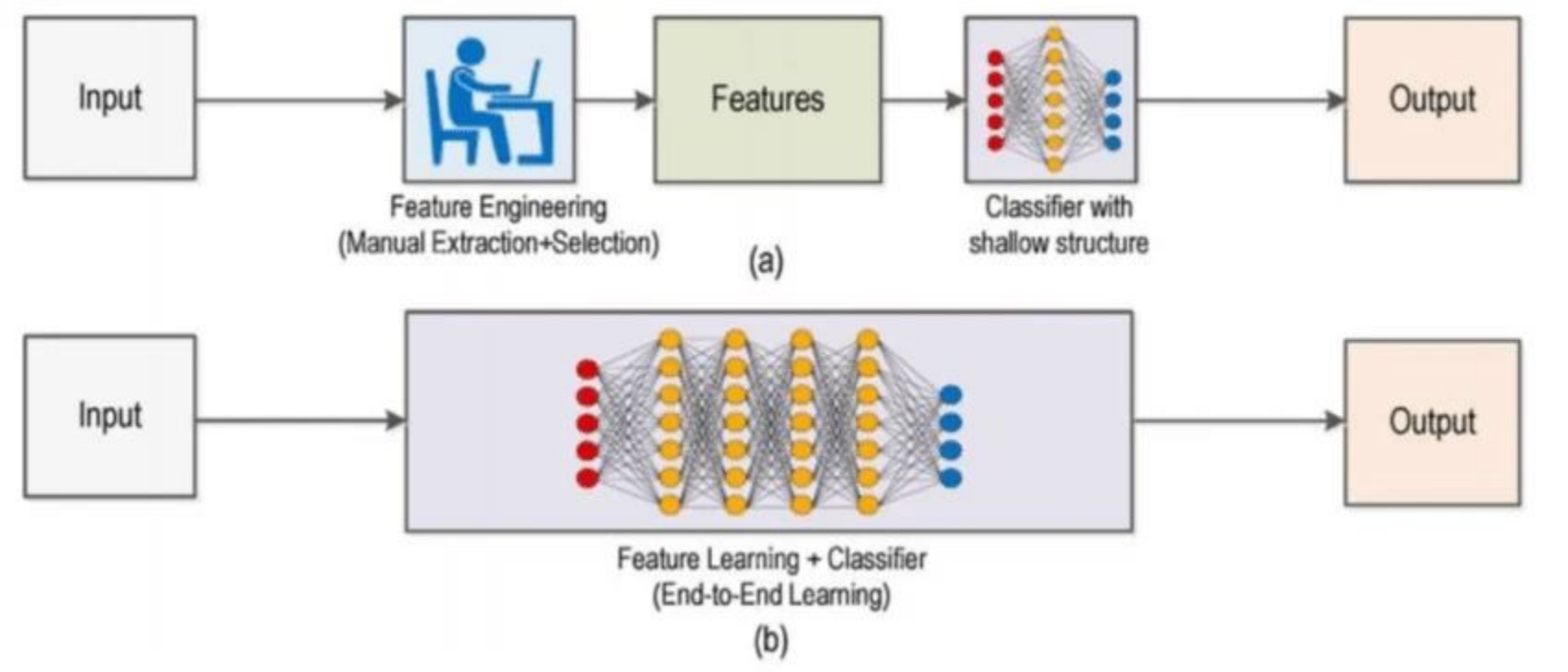
What it means

MLPs are theoretically powerful enough to learn any pattern in your data.

What it doesn't mean

"Enough neurons" may be astronomically large. In practice, deeper > wider.

Deep Learning Promise

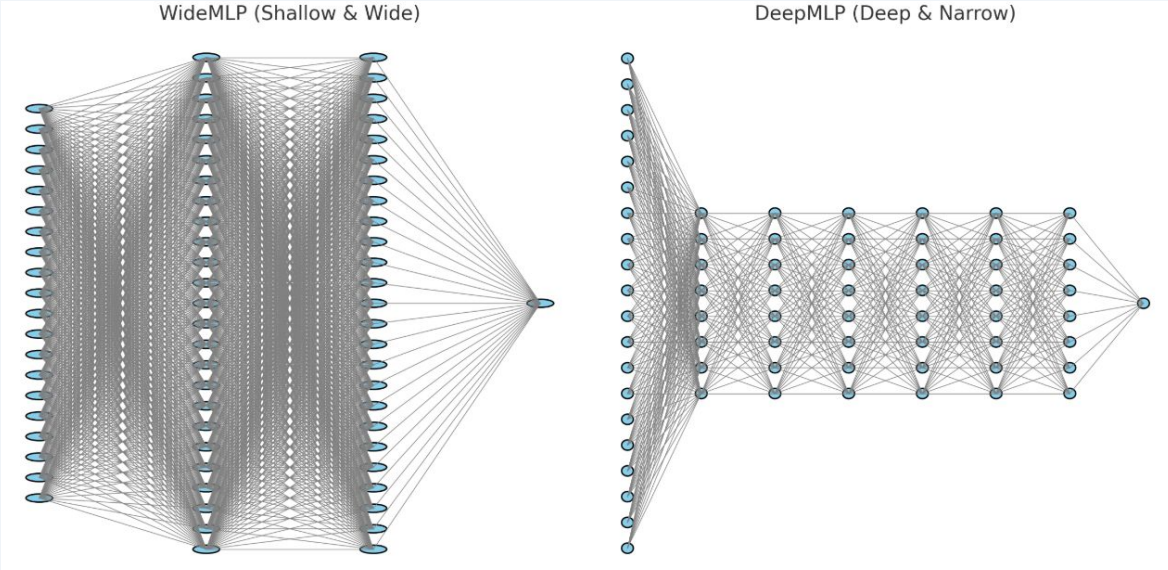


MLP architectures (hyperparameters)

Number of Layers:
Depth of the network; more layers allow learning hierarchical representations but increase training difficulty.

Number of neurons per Layers:
Width of each layer; wider layers capture more features but increase computational cost and risk of overfitting.

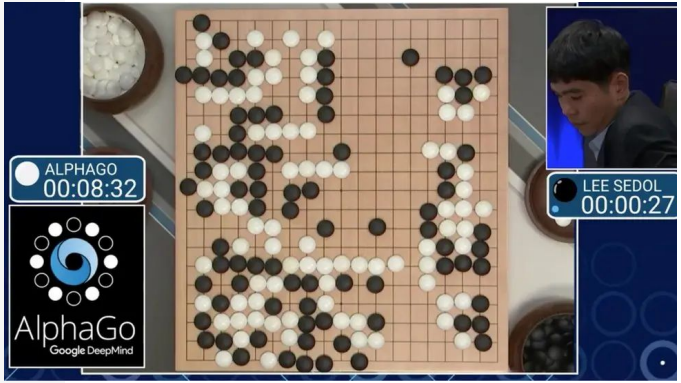
Vanishing gradient:
During backpropagation, gradients shrink exponentially through layers, making early layers learn extremely slowly or not at all.



Neural Networks became a standard

Key advantages:

- **Modularity:** very varied and adaptable architectures
- **Parallelization:** fast and efficient training
- **Performance:** modeling of complex information



Computer Vision

2012 — AlexNet wins ImageNet by a massive margin (15.3% vs 26.2% error)

Speech Recognition

2012 — Deep neural networks replace GMM-HMMs as the standard for acoustic modeling

Game Playing

2016 — AlphaGo defeats Lee Sedol

Machine Translation

2017 — Transformer architecture ("Attention Is All You Need") becomes the new paradigm

scai
SORBONNE CLUSTER FOR ARTIFICIAL INTELLIGENCE

Compute Gradient

Backpropagation

Compute directly the gradient of the loss with NN can be taught

$$f_\theta(X) = W^l \sigma(W^{l-1} \sigma(\dots \sigma(W^1 X)))$$

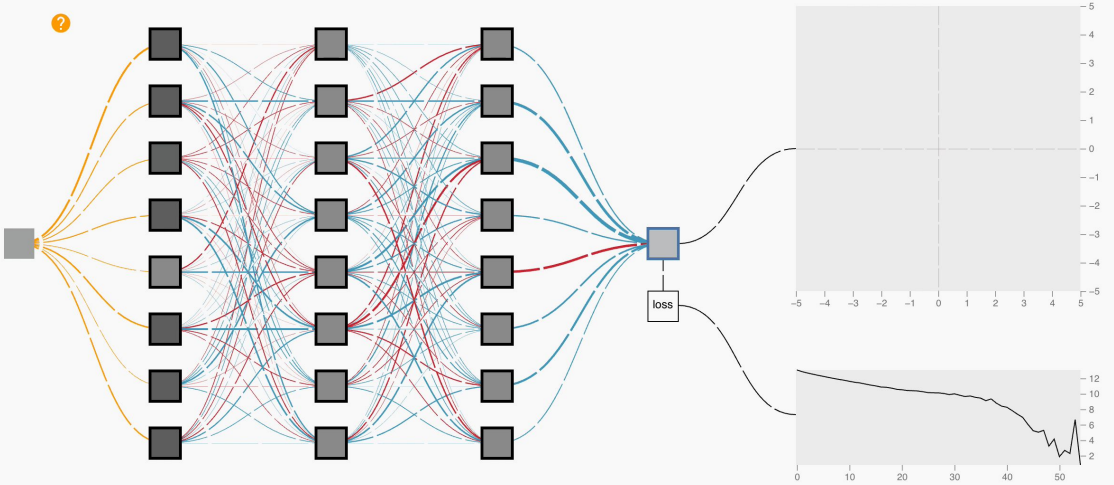
$$\nabla \ell(Y, f_\theta(X)) = \frac{\partial \ell(Y, f(X))}{\partial w_i^k} = ?$$

$$\forall k \in [0, l], \forall i \in [0, r_k]$$

Using chain rules

$$\frac{\partial z}{\partial x} = \sum_i \frac{\partial z}{\partial y_i} \frac{\partial y_i}{\partial x}$$

We are able to compute them efficiently



Step 1 : Forward Pass

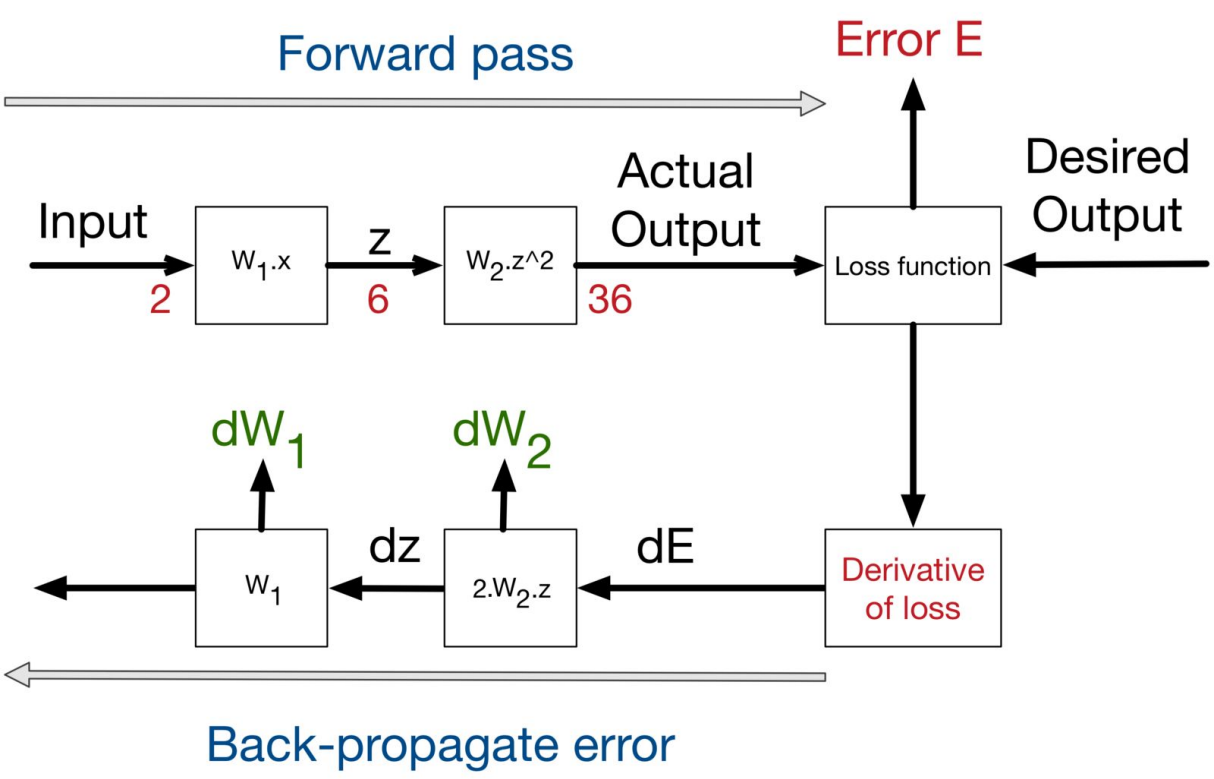
For each Layer k, compute pre-activation and activation value

Step 2 : Backward Pass

Compute gradient using value from forward and next layer gradient value

Step 3: Apply Gradient Descent

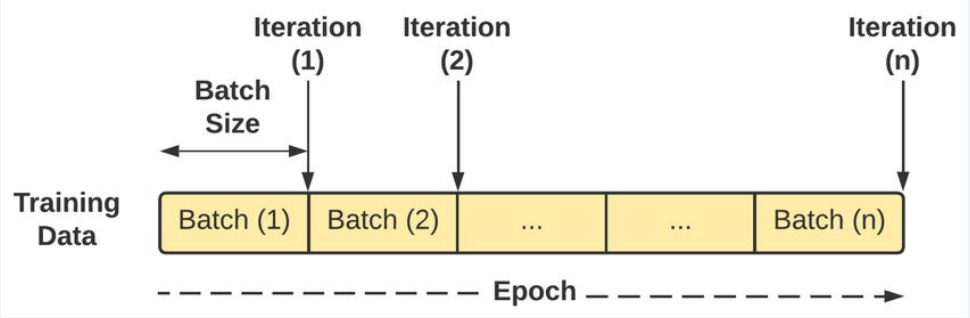
Backpropagation -> gradient update



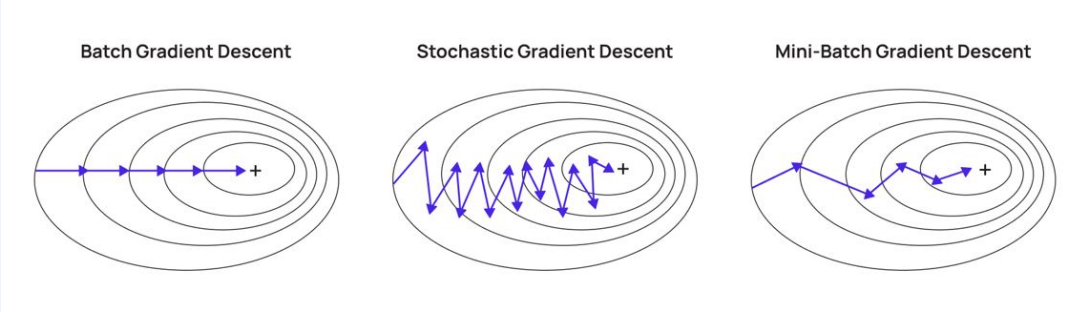
$$\theta_{t+1} = \theta_t - \eta \nabla_\theta l(\theta_t)$$

Training

Mini-batch Gradient Descent



Batch size: Number of samples processed before updating model parameters.
Epoch: One complete pass through the entire training dataset.



$$\nabla_\theta \mathcal{L} = \frac{1}{n} \sum_{i=1}^{n} \nabla_\theta \ell(f_\theta(x_i), y_i)$$

$$\nabla_\theta \mathcal{L} \approx \nabla_\theta \ell(f_\theta(x_i), y_i)$$

$$\nabla_\theta \mathcal{L} \approx \frac{1}{m} \sum_{i \in \mathcal{B}} \nabla_\theta \ell(f_\theta(x_i), y_i)$$

Optimizer

Gradient Descent:

$$w_{t+1} = w_t - \eta \nabla L(w_t)$$

Gradient Descent + Momentum:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla L(w_t)$$

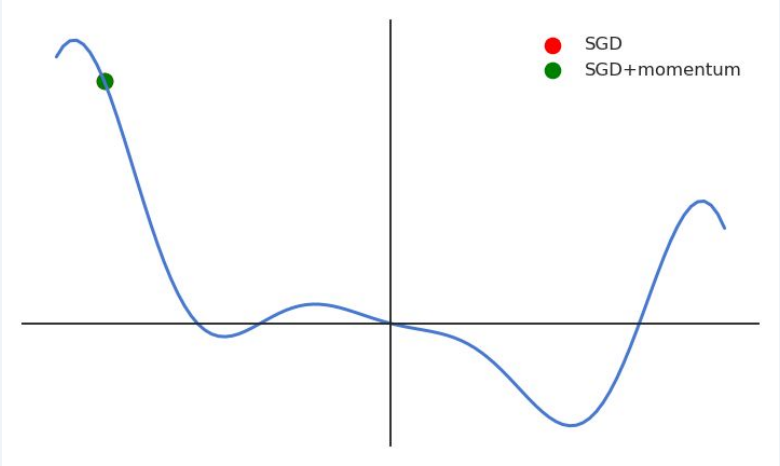
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)(\nabla L(w_t))^2$$

Adam/Adamw:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$



Quiz

1. n = 10,000 samples, batch_size = 64, 10 epochs ->
Combien d'itérations?

Quiz

1. Iterations per epoch = ceil(10000 / 64) = 157

50 epochs = 50 x 157 = 7,850 total iterations

Practical Considerations

Overfitting in Neural Networks

Neural networks have many parameters → prone to overfitting

Training loss decreases, but validation loss increases — the model memorizes instead of learning.

Early Stopping

Stop when validation loss starts increasing. Use best epoch.

Reduce Complexity

Fewer layers, fewer neurons. Smaller model = less risk.

More Data

More training examples make it harder to memorize.

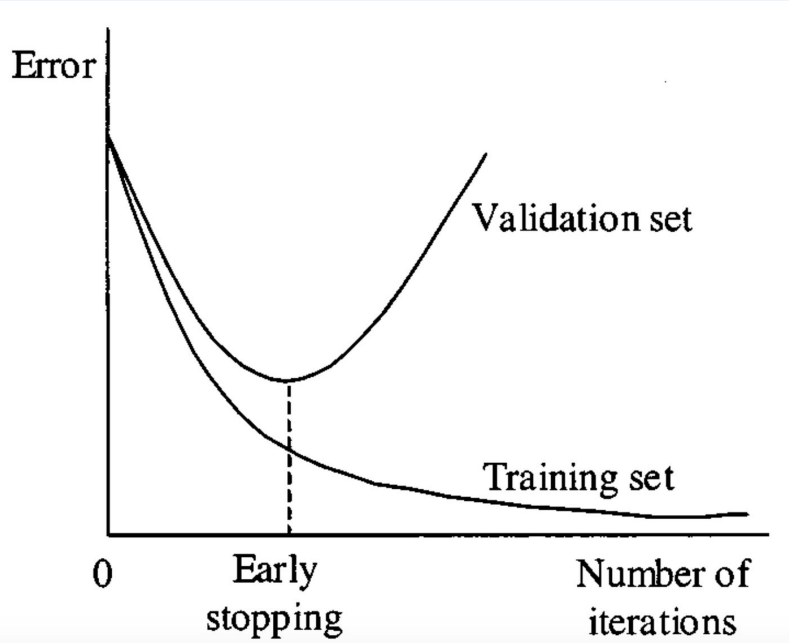
Early Stopping in Detail

How it works:

1. Split data into train + validation sets
2. Train the model, evaluate on validation after each epoch
3. Track validation loss over epochs
4. When validation loss stops improving for N epochs -> stop
5. Return the model from the best epoch

In scikit-learn:

```
MLPClassifier(early_stopping=True,
              validation_fraction=0.1,
              n_iter_no_change=10)
```

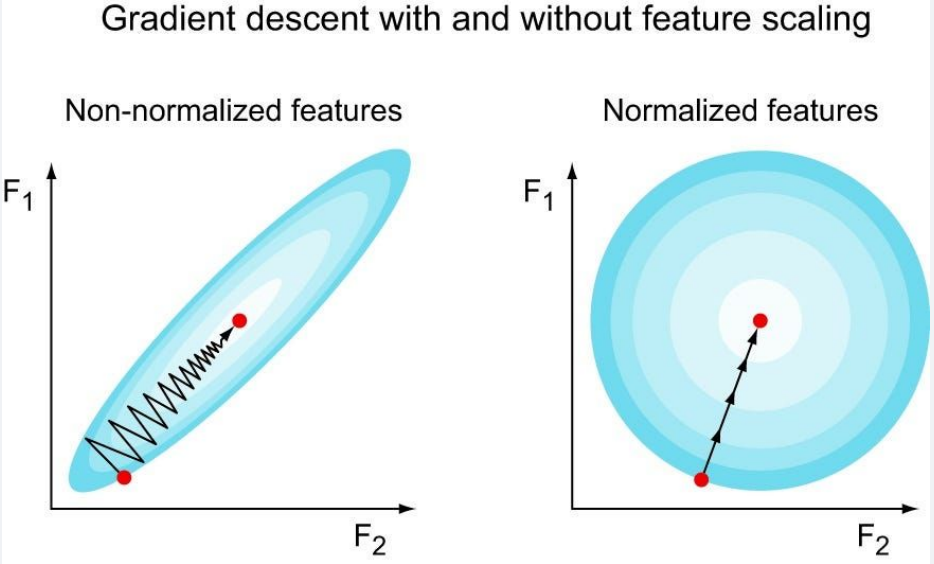


Hyperparameters to Tune

Hyperparameter	Typical values	Effect
hidden_layer_sizes	(64,), (128,64), (256,128,64)	Network capacity
activation	relu (default), tanh, logistic	Non-linearity type
learning_rate_init	0.001 (default), 0.01, 0.0001	Step size
solver	adam (default), sgd	Optimizer
max_iter	200 (default), 500, 1000	Training duration
alpha	0.0001 (default), 0.001, 0.01	L2 regularization
early_stopping	True / False	Prevent overfitting

Data Scaling is Critical for MLPs

Always scale before MLP



scai

Quiz

1. What does backpropagation compute?

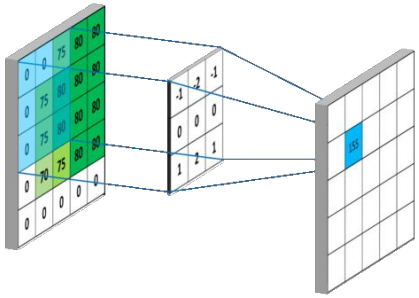
2. Why must we scale features before using MLPClassifier?

3. How does early stopping help prevent overfitting?

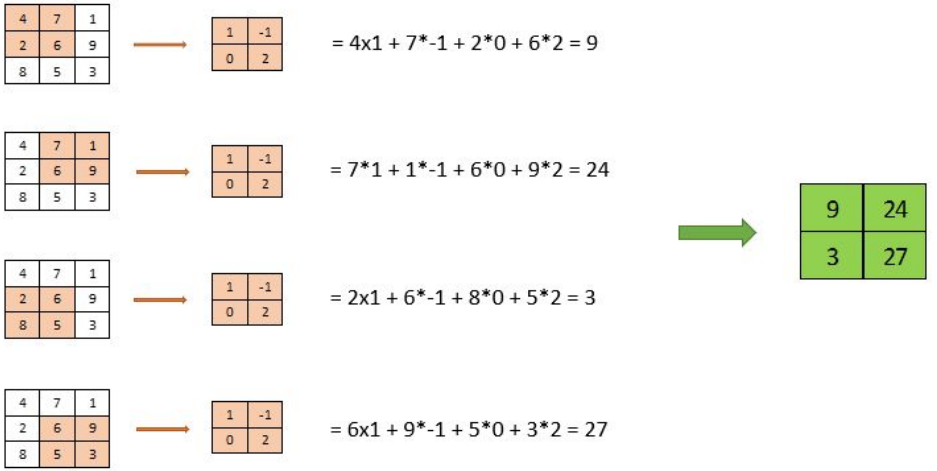
From MLP to Other Architectures

Convolution

A convolution operation **slides a kernel** (small matrix) across the image, computing weighted sums of local regions.



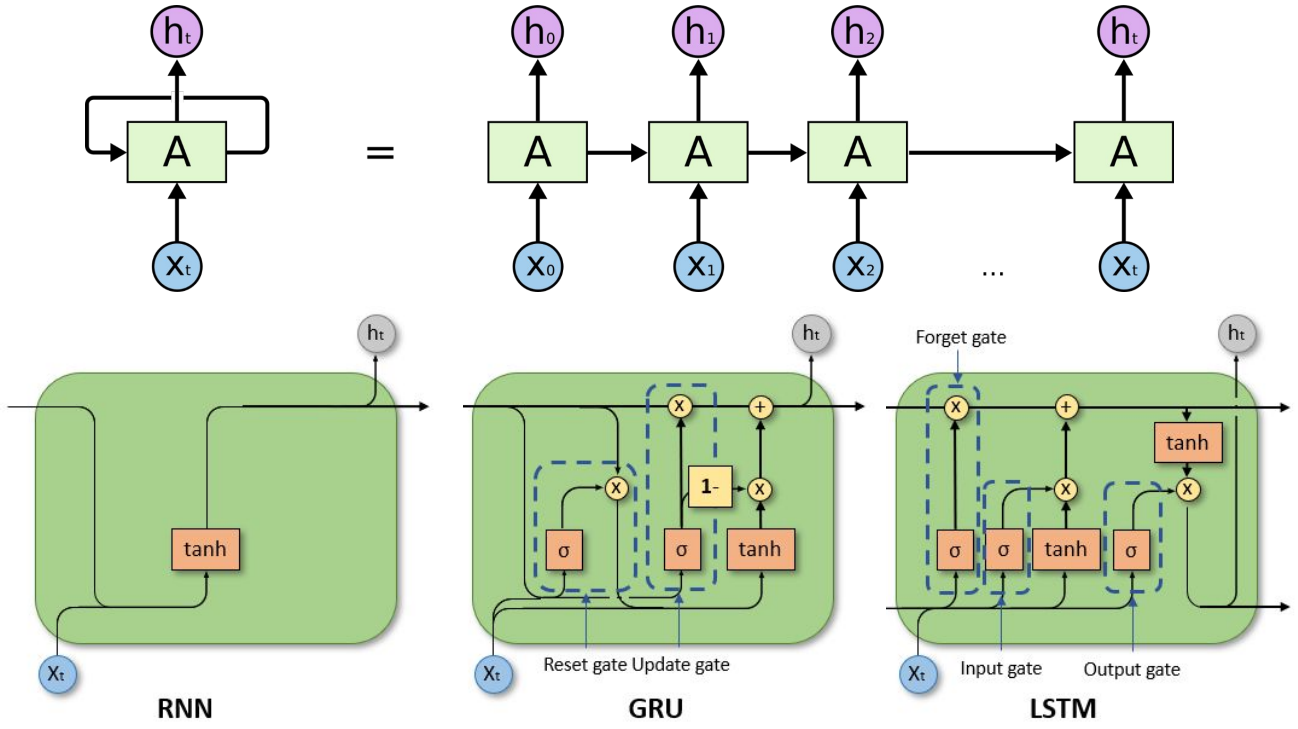
Sliding kernel



Recurrent Layers

A layer where the **output** at each step is **fed back as input** to the next step, allowing information to flow across a sequence.

In practice: slow to train (as they are sequential) and difficulties with very long sequences

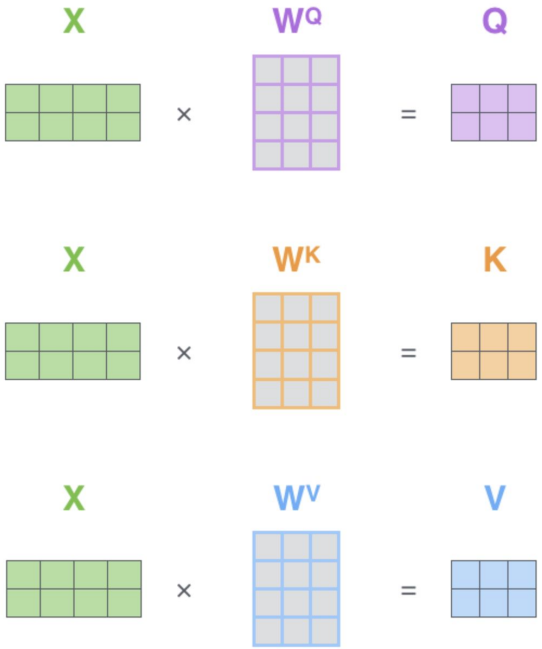


Sequence to Sequence Learning with Neural Networks

Ilya Sutskever, Oriol Vinyals, Quoc V. Le **2014**

Attention Layer

n=2, d=4

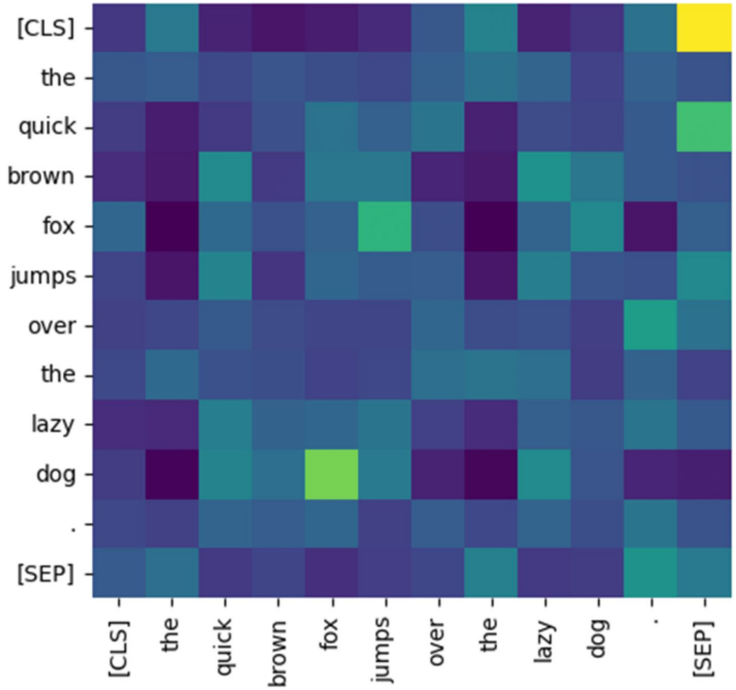


$$y = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

QK measures **how much token i wants to attend to token j**.

Softmax to sum to 1 per row

V represents the **content/information** that token j carries.



Output dimension = Input dimension

$$X_1 \in \mathbb{R}^{n_1 \times d} \xrightarrow{Q_1 K_1^T} \mathbb{R}^{n_1 \times n_1} \xrightarrow{\times V_1} \mathbb{R}^{n_1 \times d}$$

$$X_2 \in \mathbb{R}^{n_2 \times d} \xrightarrow{Q_2 K_2^T} \mathbb{R}^{n_2 \times n_2} \xrightarrow{\times V_2} \mathbb{R}^{n_2 \times d}$$

scai
SORBONNE CLUSTER FOR ARTIFICIAL INTELLIGENCE

From MLP to Deep Learning

The MLP is the foundation of all deep learning architectures.

Same core principles: layers, activations, backpropagation, gradient descent.

CNN

Convolutional layers
Specialized for images
and spatial data

RNN / LSTM

Recurrent connections
Specialized for
sequential data

Transformers

Attention mechanism
Foundation of LLMs
(GPT, BERT, ...)

MLP in scikit-learn

MLPClassifier

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

model = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(64, 32),
                  activation='relu', max_iter=500)
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Same fit() -> predict() API as every sklearn model.

MLPClassifier

```
from sklearn.neural_network import MLPRegressor

model = make_pipeline(
    StandardScaler(),
    MLPRegressor(hidden_layer_sizes=(128, 64),
                 activation='relu',
                 early_stopping=True, max_iter=1000)
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Identical API — only the import and output task change.

Evaluation: use MSE, MAE, R2 (same metrics as Course 1).

Multi-output Classification

```
model = make_pipeline(
    StandardScaler(),
    MultiOutputClassifier(
        MLPClassifier(hidden_layer_sizes=(64, 32),
                      activation='relu', max_iter=500)
    )
)
model.fit(X_train, y_train)  # y_train shape: (n_samples, n_outputs)
y_pred = model.predict(X_test)
```

Multi-output Regressor

```
model = make_pipeline(
    StandardScaler(),
    MultiOutputRegressor(
        MLPRegressor(hidden_layer_sizes=(128, 64),
                     activation='relu',
                     early_stopping=True, max_iter=1000)
    )
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```